

Day 6: NumPy

AI and Machine Learning Course

Sandesh Dhital

Nesfield International College

July 26, 2026

Today's Agenda

- 1 Introduction to NumPy
- 2 Arrays, Shapes, and Data Types
- 3 Indexing, Slicing, and Broadcasting
- 4 Mathematical Operations
- 5 Linear Algebra Basics
- 6 Practice: Matrix Operations

What is NumPy?

- **NumPy** = Numerical Python
- A Python library for working with **arrays** (collections of numbers)
- Much **faster** than regular Python lists for math
- The foundation of almost every data science library in Python

Why NumPy instead of Python Lists?

- All elements are the same type → uses less memory
- Math on entire arrays at once → no need to write loops
- Can do math on arrays of different sizes automatically

Installing and Importing NumPy

- Install with: `pip install numpy`
- Import as: `import numpy as np`
- Everyone uses the short name `np` – follow this convention

Already Installed?

If you installed Anaconda, NumPy is already available. Just import it.

What is a NumPy Array?

- A **grid of numbers** – can be 1D (a row), 2D (a table), or more
- Every element must be the **same type** (all integers, all decimals, etc.)
- Created from Python lists, or using built-in functions

Ways to Create Arrays

Method	What it does
<code>np.array([...])</code>	Convert a list to an array
<code>np.zeros((rows, cols))</code>	Array filled with 0s
<code>np.ones((rows, cols))</code>	Array filled with 1s
<code>np.arange(start, stop, step)</code>	Numbers with a step size
<code>np.linspace(start, stop, n)</code>	n evenly spaced numbers
<code>np.eye(n)</code>	Identity matrix ($n \times n$)
<code>np.random.rand(rows, cols)</code>	Random decimals between 0 and 1

Shape, Size, and Dimensions

Every array has these properties:

Property	Meaning	Example (3×4 table)
<code>.shape</code>	Rows and columns	(3, 4)
<code>.ndim</code>	Number of dimensions	2
<code>.size</code>	Total number of elements	12
<code>.dtype</code>	Type of each element	int64

Reshaping – Rearranging the Layout

- `.reshape(rows, cols)` – change to a new shape (total elements must stay the same)
- `.flatten()` – convert any array to a single row
- `.T` – swap rows and columns (transpose)

Data Types

- All elements in one array are the **same type**
- You can choose the type when creating an array, or convert later

Type	What it stores
int32, int64	Whole numbers (4 or 8 bytes each)
float32, float64	Decimal numbers (4 or 8 bytes each)
bool	True or False

Why does this matter?

Using the right type saves memory and avoids errors. For example, storing 1 million decimals as float32 instead of float64 uses half the memory.

Accessing Elements (Indexing)

1D Arrays:

- Same as Python lists
- First element: index 0
- Last element: index -1

2D Arrays:

- Use [row, column]
- Row 0, Column 0 = top-left
- Negative indices work too

Filtering (Boolean Indexing):

- Apply a condition to the array
- Returns only elements that satisfy the condition
- Example: “give me all values greater than 5”

Fancy Indexing:

- Pass a list of positions
- Returns elements at those specific positions

Getting Portions (Slicing)

- Format: `[start : stop : step]`
- `start` = where to begin (included)
- `stop` = where to end (NOT included)
- `step` = skip size (default 1)

2D Slicing

- `array[:2]` – first 2 rows
- `array[:, 1]` – all rows, second column only
- `array[0:2, 1:3]` – rows 0–1, columns 1–2 (a sub-table)

Important

Slices are **views**, not copies. Changing a slice also changes the original array!

Broadcasting (Auto-Expanding)

- NumPy can do math on arrays of **different sizes**
- It automatically “stretches” the smaller one to match the bigger one

Examples

- **Array + Number**: the number is added to every element
- **Table + Row**: the row is added to every row of the table
- **Table + Column**: the column is added to every column of the table

The Rule

For broadcasting to work, the sizes must either be **equal** or one of them must be **1** (and it gets repeated to match).

Basic Math on Arrays

Arithmetic (element by element):

- Addition: $a + b$
- Subtraction: $a - b$
- Multiplication: $a * b$
- Division: a / b
- Power: $a ** 2$

These work on **every element at once** – no loops needed!

Built-in Math Functions:

- `np.sqrt()` – square root
- `np.exp()` – e^x
- `np.log()` – natural log
- `np.sin()`, `np.cos()` – trigonometry
- `np.abs()` – absolute value

Conditional:

- `np.where(condition, yes, no)`

Summary Functions

These give you a **single answer** from an entire array:

Function	What it returns
<code>np.sum()</code>	Total of all elements
<code>np.mean()</code>	Average
<code>np.std()</code>	Spread (standard deviation)
<code>np.min()</code> / <code>np.max()</code>	Smallest / Largest value
<code>np.argmin()</code> / <code>np.argmax()</code>	Position of smallest / largest

axis parameter

- `axis=0` → work down each column (one answer per column)
- `axis=1` → work across each row (one answer per row)
- No axis → work on all elements (one single answer)

Mathematical Functions

Category	Function	What it does
Math	<code>np.sqrt()</code>	Square root
	<code>np.exp()</code>	e raised to the power
	<code>np.log()</code> / <code>np.log10()</code>	Natural log / base-10 log
	<code>np.power(a, n)</code>	Raise to n -th power
	<code>np.abs()</code>	Absolute value
Trigonometry	<code>np.sin()</code> / <code>np.cos()</code>	Sine / Cosine
	<code>np.tan()</code>	Tangent
	<code>np.arcsin()</code>	Inverse sine
	<code>np.degrees()</code> / <code>np.radians()</code>	Convert units
Rounding	<code>np.round(a, n)</code>	Round to n decimals
	<code>np.floor()</code>	Round down
	<code>np.ceil()</code>	Round up

Statistical Functions

Category	Function	What it does
Central Tendency	<code>np.mean()</code>	Average value
	<code>np.median()</code>	Middle value (sorted)
	<code>np.average(a, weights)</code>	Weighted average
	<code>np.percentile(a, q)</code>	Value at q -th percentile
Spread	<code>np.std()</code>	Standard deviation
	<code>np.var()</code>	Variance (std^2)
	<code>np.ptp()</code>	Range ($\text{max} - \text{min}$)
Other	<code>np.corrcoef()</code>	Correlation between arrays
	<code>np.histogram()</code>	Frequency counts in bins
	<code>np.unique()</code>	Unique values
	<code>np.cumsum()</code>	Running total

Other Useful Functions

Category	Function	What it does
Sorting & Searching	<code>np.sort()</code>	Sort elements
	<code>np.argsort()</code>	Positions that would sort it
	<code>np.where()</code>	Find positions where condition is True
	<code>np.argmax()</code> / <code>np.argmin()</code>	Position of largest / smallest
Shape Manipulation	<code>np.reshape()</code>	Change dimensions
	<code>np.concatenate()</code>	Join arrays together
	<code>np.split()</code>	Break array into pieces
	<code>np.stack()</code> / <code>np.vstack()</code>	Stack arrays vertically/horizontally
Logic	<code>np.all()</code>	Are ALL elements True?
	<code>np.any()</code>	Is ANY element True?
	<code>np.isnan()</code>	Check for missing values

Linear Algebra – Why Does It Matter?

- **Linear algebra** = math with matrices (tables of numbers)
- Used heavily in Machine Learning:
 - Data is stored as matrices (rows = samples, columns = features)
 - Models learn by doing matrix operations

Key Operations You'll Use

Operation	What it does
Matrix Multiply (@)	Combines two matrices (not element-wise!)
Transpose (.T)	Swaps rows and columns
Determinant	A single number: can we invert this matrix?
Inverse	"Undo" a matrix (like dividing)
Solve	Find unknowns in a system of equations

Matrix Multiplication

- **NOT** the same as element-wise multiply ($*$)
- Uses the $@$ operator or `np.dot()`
- Rule: number of **columns in A** must equal number of **rows in B**

How It Works

Each element in the result = dot product of a row from A and a column from B.

If A is $(m \times n)$ and B is $(n \times p)$, then $A @ B$ is $(m \times p)$.

Example

A is 2×2 , B is $2 \times 2 \rightarrow$ Result is 2×2

A is 3×2 , B is $2 \times 4 \rightarrow$ Result is 3×4

Solving Equations with NumPy

- You can solve systems of equations without doing algebra by hand
- Put the coefficients in a matrix, the answers in a list
- NumPy finds the values of the unknowns

Example

- $2x + 3y = 8$
- $4x + y = 5$

Put coefficients $\begin{pmatrix} 2 & 3 \\ 4 & 1 \end{pmatrix}$ and answers $\begin{pmatrix} 8 \\ 5 \end{pmatrix}$ into NumPy.

It returns: $x = 0.7, y = 2.2$

Function

`np.linalg.solve(A, b)` – solves the system $Ax = b$

Practice Exercises

- 1 Create a 3×3 matrix with values 1–9. Find its transpose and determinant.
- 2 Generate a random 4×4 matrix. Find the sum of each row, average of each column, and overall spread (std).
- 3 Solve: $3x + 2y = 12$, $x - y = 1$
- 4 Create two matrices and check that $(AB)^T = B^T A^T$
- 5 Normalize a matrix: for each column, subtract its average and divide by its spread.

Summary

- **Arrays:** Fast containers for numbers (1D, 2D, or more)
- **Shape & dtype:** How many rows/columns, and what type of number
- **Indexing/Slicing:** Pick out elements or portions you need
- **Broadcasting:** Math on arrays of different sizes (auto-expands)
- **Math operations:** Apply to whole arrays at once, no loops needed
- **Linear algebra:** Multiply matrices, find inverses, solve equations

Next: Day 7 – Pandas Part 1

Tables of real data, reading CSV files, filtering and sorting